Altrixs Staff Portal — Screen Inventory & Daily Developer Prompts

Generated for handoff to developers · Repo: `divinecare-platform` (Altrixs Next.js) · Goal: every nav screen working end-to-end (UI + API + Prisma + workflow) · Cadence: **1 screen / day** (UI refine → backend → connect workflow → test)

How to use this doc: Give the developer *exactly one* Day prompt per day (copy the Prompt box). After they finish, run the Test checklist yourself. Do not skip Days 1–8 — later screens depend on clients, appointments, and notes.

1. Status legend

Badge	Meaning
LIVE	UI talks to real `/api/*` + Prisma. Create/update persists.
PARTIAL	Some tabs/actions live; other sections still mock or empty stubs.
STUB HOOK	Feature hook exists but returns empty / throws until API is built. Page may still show DEMO arrays.
UI ONLY	Hardcoded `DEMO_*` data. No real API. Not workflow-connected.

2. What is already LIVE (do not rebuild)

Route	Status	Backend evidence	Roles
`/login`, auth OTP/reset/setup	LIVE	`/api/auth/*`, NextAuth credentials	All
`/dashboard`	LIVE	`/api/dashboard/overview`	Clinic roles
`/dashboard/clients`	LIVE	`/api/clients`, intake modal	Admin, Director, Clinician, Front Desk, Billing
`/dashboard/clients/[id]`	PARTIAL	Client fetch + avatar live; Notes tab local-only; appointments list may be thin	Same
`/dashboard/appointments`	LIVE	`/api/appointments` CRUD/status	Admin, Director, Clinician, Front Desk
`/dashboard/hr` (staff invite)	LIVE	`/api/staff`, invite/accept/resend	Admin, Director, HR
`/dashboard/admin`	PARTIAL	Users + Locations live; HIPAA audit / security / integrations = mock tabs	Admin, Director
`/dashboard/settings`	LIVE	`/api/profile`, avatar	All clinic
`/platform`, orgs, users	LIVE	`/api/platform/orgs`, `/api/platform/users`	Super Admin

Prisma models that already exist: Organization, Location, User, Client, Appointment, CheckIn, RiskAlert, Message, EmailOtp.

Missing Prisma domains (need new models for later days): WaitlistEntry, IntakePacket, ClinicalNote, TreatmentPlan, Consent, Eligibility, PriorAuth, Claim/Payment, Credential, SupervisionLog, TelehealthSession, Task/WorkItem, Referral, Survey, etc.

3. Full screen inventory (UI-only / not connected)

#	Route	Status	Priority wave	Primary roles
1	`/dashboard/clients/waitlist`	UI ONLY	Wave A — Front Desk	Front Desk, Admin
2	`/dashboard/clients/intake`	UI ONLY	Wave A	Front Desk
3	`/dashboard/messages`	UI ONLY + STUB HOOK	Wave A	FD, Clinician, HR
4	`/dashboard/alerts`	UI ONLY + STUB HOOK	Wave A	Clinician
5	`/dashboard/tasks`	UI ONLY	Wave A	Most clinic

6	`/dashboard/telehealth`	UI ONLY	Wave B — Clinical	Clinician
7	`/dashboard/ehr` overview	UI ONLY	Wave B	Clinician
8	`/dashboard/ehr/notes`	UI ONLY	Wave B	Clinician
9	`/dashboard/ehr/treatment-plans`	UI ONLY	Wave B	Clinician
10	`/dashboard/ehr/assessments`	UI ONLY	Wave B	Clinician
11	`/dashboard/ehr/diagnoses`	UI ONLY	Wave B	Clinician
12	`/dashboard/ehr/consents`	UI ONLY	Wave B	FD + Clinician
13	`/dashboard/ehr/eligibility`	UI ONLY	Wave C — Billing ops	Billing + FD
14	`/dashboard/ehr/prior-auths`	UI ONLY	Wave C	Billing + FD
15	`/dashboard/ehr/prescriptions`	UI ONLY	Wave B+	Clinician
16	`/dashboard/ehr/group-sessions`	UI ONLY	Wave B+	Clinician
17	`/dashboard/ehr/supervisor-review`	UI ONLY	Wave B+	Director
18	`/dashboard/ehr/session-ai` , note-builder, goal-bank, part2	UI ONLY	Wave E — later	Clinician
19	`/dashboard/scribe` , ai-insights, coding, mbc*	UI ONLY	Wave E	Clinician / Billing
20	`/dashboard/billing` + denials/payments/autopay/superbills/panels	UI ONLY	Wave C	Billing
21	`/dashboard/hr/credentialing` , supervision	UI ONLY	Wave D — HR	HR
22	Analytics, reports, resources, community, PR, PRP, enterprise, subscription	UI ONLY	Wave E	Leadership
23	`/book/[practiceSlug]`	UI ONLY	Wave E	Public

Approx count: ~8 LIVE/PARTIAL core routes · ~40+ UI-only feature screens. Prisma already has `RiskAlert` + `Message` models but pages/hooks are not wired.

4. Recommended delivery order (why)

1. **Days 1–2:** Wire Client Detail gaps + Waitlist (Front Desk intake funnel)
2. **Days 3–4:** Digital Intake + Consents (eligibility for first appointment)
3. **Days 5–6:** Messages + Risk Alerts (models exist — fastest wins)
4. **Days 7–8:** Clinical Notes + Treatment Plans (clinician core EHR)
5. **Days 9–10:** Tasks (derived from unsigned notes / expiring PA) + Telehealth link to appointments
6. **Days 11–13:** Eligibility, Prior Auths, Billing claims spine
7. **Days 14–15:** HR credentialing + supervision
8. **Days 16+:** AI/analytics shells — only after core workflow is green

5. Global rules for every Day prompt

- Keep NextAuth + `roleAccess.ts` enforcement. Add server-side `canAccessPath` in `proxy.ts` when touching authz.
- Remove `DEMO_*` arrays from the target page; load from API. Empty states OK.
- Add Prisma models + migration via `prisma db push` (or migrate) scoped to that domain.
- Match Amanah-style clinic UI already used on Waitlist/To-Do (soft cards, pills, navy/purple).
- Wire workflow links: e.g. Waitlist “Convert to Intake” → create Client; Appointment COMPLETED → create unsigned Note task.
- Do not invent third-party integrations (Availity, Zoom) — stub provider interface with local persistence.
- After code: `npx tsc --noEmit` must pass.

Local test logins (password `Altrixs@123`):

- `admin@altrixs.local` · `clinician@altrixs.local`
- `frontdesk@altrixs.local` · `billing@altrixs.local` · `hr@altrixs.local` · `director@altrixs.local`

6. Daily prompts (copy-paste one per day)

DAY 1 — Client detail completion

PARTIAL → LIVE

Route: `/dashboard/clients/[id]` · **Roles:** Clinician, Front Desk, Admin

```
Day 1 — Finish Client Detail so every tab is backend-backed.

Context: `/dashboard/clients/[id]` already loads client via `/api/clients/[id]` and avatar upload works. Notes tab
is local-only. Appointments/check-ins may be incomplete.

Do:
1) UI polish to match clinic design system (PageHeader style, soft cards).
2) Persist clinical notes on the client (Prisma: add ClientNote model OR reuse Message with type — prefer ClientNote
with authorId, body, createdAt).
3) Appointments tab: list this client's appointments from existing Appointment model via API.
4) Check-ins tab: read CheckIn rows for client; allow staff to add a check-in (mood/stress/note).
5) Insurance tab: edit insurance fields already on Client; PATCH `/api/clients/[id]`.
6) Enforce roleAccess; Clinician only sees assigned clients if already gated.

Acceptance: create note → refresh → still there. Add check-in → chart updates. Edit insurance → persists.
```

How to test after Day 1

1. Login as `clinician@altrixs.local` → Clients → open a client.
2. Notes: add note → hard refresh → note remains with author/timestamp.
3. Check-Ins: add mood/stress → appears in list/chart.
4. Insurance: change payer → save → reload → same value.
5. Appointments tab shows real rows if that client has appointments (create one from Appointments page first).
6. `npx tsc --noEmit` clean.

DAY 2 — Waitlist Manager

UI ONLY → LIVE

Route: `/dashboard/clients/waitlist` · **Roles:** Front Desk, Admin, Director

```
Day 2 — Make Waitlist Manager fully working (API + Prisma + workflow).

Context: `/dashboard/clients/waitlist` is DEMO-only. UI was refined (table + detail panel). Keep that UI; remove
DEMO.

Do:
1) Prisma model WaitlistEntry (clientName, contact, specialty, urgency, status waiting|matched|scheduled|closed,
insurance, referralSource, notes, daysWaiting or addedAt, clinicianId optional, organizationId).
2) APIs: GET/POST `/api/waitlist`, PATCH `/api/waitlist/[id]` (status, match clinicianId), soft-close.
3) Match flow: assign clinician from Staff list (existing `/api/staff`).
4) Workflow: "Convert to Intake" / matched → create Client (or link) + optional draft Appointment; update status
scheduled.
5) Wire page to APIs; empty states; roleAccess already allows Front Desk.

Acceptance: Front Desk can add entry, match clinician, convert to client, see it on Clients list.
```

How to test after Day 2

1. Login `frontdesk@altrixs.local` → Waitlist.
2. Add entry → appears in table after reload.
3. Match to clinician → status Matched; assigned name shows.
4. Convert to intake/client → new client in `/dashboard/clients`.
5. Login as that clinician → client visible if assignment rules apply.
6. Billing user should NOT see waitlist in nav (roleAccess).

DAY 3 — Digital Intake

UI ONLY → LIVE

Route: `/dashboard/clients/intake`

```
Day 3 — Digital Intake packets connected to Clients + Waitlist.

Do:
1) Prisma IntakeInvite (token, clientId optional, email, status pending|completed|expired, expiresAt, formPayload
JSON).
2) APIs to create invite, mark complete, list by org.
3) Completing intake creates/updates Client demographics + insurance fields.
4) Link from Waitlist "Convert to Intake" creates invite.
5) Replace DEMO on intake page; Amanah-style list UI.

Acceptance: send/create invite → complete payload → Client row updated/created.
```

How to test after Day 3

1. Front Desk creates intake from Intake page.
2. Mark complete with demographics → Clients shows new/updated patient.
3. Expired invites show Expired status.
4. Waitlist convert creates pending intake visible on Intake page.

DAY 4 — Consents

UI ONLY → LIVE

Route: `/dashboard/ehr/consents`

```
Day 4 — Consent forms backend.

Do:
1) Prisma Consent (clientId, type, status signed|pending|expired, signedAt, documentMeta).
2) CRUD APIs scoped to org; Front Desk + Clinician can manage per roleAccess.
3) Workflow: Client detail shows consent summary; block "first appointment check-in" if required consents missing
(soft warning OK).
4) Remove DEMO_CONSENTS; polish UI.

Acceptance: create pending consent → mark signed → persists; shows on client overview.
```

How to test after Day 4

1. Create consent for a client → pending.
2. Sign/mark signed → reload still signed.
3. Client detail shows consent status.
4. HR user cannot open consents (nav hidden / redirect).

DAY 5 — Secure Messages

UI ONLY → LIVE

Route: `/dashboard/messages` · Prisma `Message` already exists

```
Day 5 — Wire Messages page to real Message model.

Context: `prisma` has Message (clientId, sender STAFF|CLIENT, content, readAt). `features/messages` stubs return [].
Page uses DEMO_THREADS.

Do:
1) Implement `/api/messages/threads` and `/api/messages` (list by client, send as STAFF, mark read).
2) Replace stubs in messagesApi.ts; remove DEMO from page.
3) UI: thread list = clients with messages; compose sends to API.
4) Optional: unread badge in DashboardShell later.

Acceptance: send message as staff → appears after refresh; client-scoped threads.
```

How to test after Day 5

1. Clinician opens Messages → pick/create thread for a client.
2. Send "Hello" → refresh → message still there.
3. Second staff in same org can see thread if product rule allows (document rule).
4. Network tab shows `/api/messages` 200, not DEMO.

DAY 6 — Risk Alerts

UI ONLY → LIVE

Route: `/dashboard/alerts` · Prisma `RiskAlert` exists

```
Day 6 — Wire Risk Alerts to RiskAlert model.

Do:
1) `/api/risk-alerts` GET (filter unreviewed), POST (create from check-in thresholds optional), PATCH review.
2) Implement alertsApi stubs; remove DEMO_ALERTS from page.
3) Workflow: when CheckIn mood <= 3 or stress >= 8, auto-create RiskAlert (hook from Day 1 check-in API).
4) Review marks reviewed=true + optional note.

Acceptance: create check-in that triggers alert → appears on Alerts; review removes from unreviewed.
```

How to test after Day 6

1. Add check-in mood=2 for a client.
2. Alerts page shows new high/medium alert.
3. Review alert → unreviewed count drops; refresh persists.
4. Front Desk should not see Alerts (roleAccess).

DAY 7 — Progress Notes (EHR)

UI ONLY → LIVE

Route: `/dashboard/ehr/notes`

```
Day 7 — Clinical progress notes (core EHR).

Do:
1) Prisma ClinicalNote (clientId, appointmentId optional, authorId, status draft|signed, soap/content JSON or text,
signedAt).
2) Full CRUD + sign endpoint (immutable after sign).
3) Workflow: from Appointments, “Complete” suggests create note; unsigned notes feed Tasks later.
4) Replace DEMO_NOTES; list/filter/sign UI polish.
5) Client detail Notes tab should use same API as Day 1 (unify models — migrate ClientNote → ClinicalNote if
needed).

Acceptance: draft → sign → cannot edit body; appears under client.
```

How to test after Day 7

1. Clinician creates draft note for client.
2. Sign note → status signed; edit blocked.
3. Reload `/dashboard/ehr/notes` and client detail — same note.
4. Billing cannot access notes route.

DAY 8 — Treatment Plans

UI ONLY → LIVE

Route: `/dashboard/ehr/treatment-plans`

```
Day 8 — Treatment plans API + expiry workflow.

Do:
1) Prisma TreatmentPlan (clientId, goals JSON, status active|expiring|expired, startsAt, endsAt, signedById).
2) CRUD + renew endpoint.
3) Workflow: plans ending < 14 days flag for Tasks/To-Do.
4) Remove DEMO_PLANS; connect UI.

Acceptance: create plan with end date → appears; renew extends endsAt.
```

How to test after Day 8

1. Create plan ending in 7 days → shows expiring state.
2. Renew → new end date persisted.
3. Visible on client clinical summary if linked.

DAY 9 — My To-Do (derived work queue)

UI ONLY → LIVE

Route: `/dashboard/tasks`

```
Day 9 — Replace hardcoded TASKS with computed + dismissible work items.

Do:
1) Prisma WorkItem optional for dismissals OR compute live from: unsigned ClinicalNotes, expiring TreatmentPlans,
expiring PriorAuths (stub empty until Day 12), incomplete intakes.
2) GET `/api/tasks` returns unified list for current user/role.
3) POST dismiss (hide 24h).
4) Keep polished Amanah UI from recent redesign; remove static TASKS array.
5) Deep links actionHref must open real records.

Acceptance: unsigned note from Day 7 appears as urgent task; dismiss hides; new note signing clears task.
```

How to test after Day 9

1. Create unsigned draft note → To-Do shows “Unsigned Progress Note”.
2. Sign note → task gone after refresh.
3. Dismiss task → hidden; restore works if implemented.
4. Filters by type/priority work on live data.

DAY 10 — Telehealth linked to Appointments

UI ONLY → LIVE

Route: `/dashboard/telehealth`

```
Day 10 — Telehealth sessions from real virtual appointments.

Do:
1) Use Appointment.isVirtual + status; optional TelehealthSession (joinUrl stub, startedAt).
2) API lists today's virtual appointments for clinician.
3) “Start session” sets CHECKED_IN / in-progress metadata; “Complete” → COMPLETED + prompt to write note (link to
notes).
4) Remove TODAY_SESSIONS mock; polish UI.

Acceptance: create virtual appointment → appears on Telehealth today; complete → appointment COMPLETED.
```

How to test after Day 10

1. Book virtual appointment for today as Front Desk for clinician.
2. Login clinician → Telehealth shows session.
3. Start/Complete → Appointments status updates.
4. CTA to write note opens notes with client preselected if possible.

DAY 11 — Eligibility

UI ONLY → LIVE

Route: `/dashboard/ehr/eligibility`

```
Day 11 — Eligibility checks persisted (stub payer response).

Do:
1) Prisma EligibilityCheck (clientId, payer, status active|inactive|unknown, checkedAt, raw JSON stub).
2) POST runs “check” that returns deterministic stub from client insurance fields.
3) List history on page; Billing + Front Desk access.
4) Remove DEMO; link from client insurance tab “Run eligibility”.

Acceptance: run check → history row saved; client insurance payer shown.
```

How to test after Day 11

1. Billing login → Eligibility → run for client with Medicaid.
2. Result row persists after refresh.
3. Clinician may have access per roleAccess — verify matrix.

DAY 12 — Prior Authorizations

UI ONLY → LIVE

Route: `/dashboard/ehr/prior-auths`

```
Day 12 — Prior Auths CRUD + units + expiry.

Do:
1) Prisma PriorAuth (clientId, payer, authNumber, unitsTotal, unitsUsed, expiresAt, status).
2) APIs + UI; feed Day 9 tasks when expiring or units > 90%.
3) Remove DEMO_AUTHS.

Acceptance: create PA → decrement units → expiring appears on To-Do.
```

How to test after Day 12

1. Create PA expiring in 4 days.
2. To-Do shows prior auth task.
3. Update units to 90%+ → units-low task.

DAY 13 — Billing claims spine

UI ONLY → LIVE

Route: `/dashboard/billing` (core first; subpages later)

```
Day 13 — Billing home: Claim model from signed notes/appointments.

Do:
1) Prisma Claim (clientId, appointmentId optional, noteId optional, cpt, amount, status
draft|submitted|approved|denied|pending, payer).
2) “Generate draft claims” from completed appointments missing claims.
3) Submit stub flips status submitted→pending.
4) Replace STATS/CLAIMS mocks on billing page only (denials/payments can stay UI until Days 16–17).

Acceptance: complete appointment + signed note → draft claim → submit persists.
```

How to test after Day 13

1. Complete appointment + sign note for client.
2. Billing → generate/view draft claim.
3. Submit → status pending after refresh.
4. Clinician cannot open billing.

DAY 14 — HR Credentialing

UI ONLY → LIVE

Route: `/dashboard/hr/credentialing`

```
Day 14 — Staff credentials tied to User.

Do:
1) Prisma Credential (userId, type, expiresAt, status).
2) CRUD for HR; alerts for expiring_soon/expired.
3) Optionally sync User.licenseExpiresAt.
4) Remove DEMO.

Acceptance: add malpractice expiring in 9 days → shows in alerts list.
```

How to test after Day 14

1. HR login → add credential for clinician.
2. Expiring filter works; refresh persists.
3. Front Desk cannot access credentialing.

DAY 15 — Supervision logs

UI ONLY → LIVE

Route: `/dashboard/hr/supervision`

```
Day 15 — Supervision logs for supervisee clinicians.

Do:
1) Prisma SupervisionLog (superviseeId, supervisorId, sessionDate, notes, signed flags).
2) APIs; Clinician can view own logs; HR/Director manage.
3) Remove DEMO; connect to real staff users.

Acceptance: create log → supervisee sees it; signature fields persist.
```

How to test after Day 15

1. Director/HR creates supervision log for clinician.
2. Clinician login sees log under supervision.
3. Sign/ack persists.

DAY 16+ — Remaining UI shells (batch prompts)

UI ONLY

Give these only after Days 1–15 are green. One screen per day still.

```
Template for Days 16+:
“Implement LIVE backend for `/dashboard/<route>`.
Remove DEMO data. Add Prisma model + `/api/...` CRUD.
Connect workflow to Clients/Appointments/Notes/Claims as relevant.
Match clinic UI. Enforce roleAccess.
Include seed data for local demo.
Done when: create → reload → still there; wrong roles blocked.”

Suggested order after core:
16 Billing payments · 17 Denials · 18 Autopay · 19 Superbills · 20 Payer panels ·
21 Diagnoses · 22 Assessments · 23 Prescriptions · 24 Group sessions · 25 Supervisor review ·
26 EHR overview aggregates · 27 Coding · 28 MBC · 29 Outcomes · 30 Scribe (local only) ·
31 Analytics schedule · 32 Productivity · 33 Surveys · 34 Reports ·
35 Resources · 36 Community events · 37 Referrals · 38 PR · 39 PRP · 40 Enterprise/Subscription polish ·
41 Public booking `/book/[slug]` → creates WaitlistEntry or Appointment ·
42 Admin mock tabs (audit log) · 43 Server-side role path enforcement in proxy.ts ·
44 End-to-end workflow QA day (script below)
```

Day 16+ per-screen test (generic)

1. Correct role: open screen, create record, hard refresh, record remains.
2. Wrong role: nav hidden + direct URL redirects home.
3. Related workflow link works (e.g. claim ↔ client).
4. `npx tsc --noEmit` + no console errors on happy path.

7. End-to-end clinic workflow QA (after Day 15)

1. **Front Desk:** Waitlist add → match → intake complete → client created → book appointment.
2. **Front Desk:** Consent pending → signed; eligibility check run.
3. **Clinician:** Check-in patient → telehealth/start → complete appointment → write+sign note.
4. **System:** Risk alert from low mood; To-Do clears unsigned note after sign.
5. **Billing:** Draft claim from completed visit → submit.
6. **HR:** Credential + supervision log for clinician.
7. **Admin:** Invite staff; locations still work.
8. **Super Admin:** `/platform` orgs only; cannot use clinic dashboard.

8. Existing API map (already in repo)

`/api/auth/*` · `/api/clients` · `/api/appointments` · `/api/staff` (+ invite) · `/api/locations` · `/api/org` · `/api/profile` · `/api/dashboard/overview` · `/api/platform/*`

Needed new API families: waitlist, intake, consents, messages (wire), risk-alerts (wire), clinical-notes, treatment-plans, tasks, telehealth, eligibility, prior-auths, claims, credentials, supervision, …